

Capstone Project: E-Commerce Database Design

Simplified Flipkart-like Online Store

Table of Contents

1. [Project Overview](#)
 2. [Database Schema \(Visual\)](#)
 3. [Table Structure & Sample Data](#)
 4. [CREATE TABLE Statements](#)
 5. [INSERT Sample Data](#)
 6. [Day 2 Query Examples](#)
-

Project Overview

What We're Building:

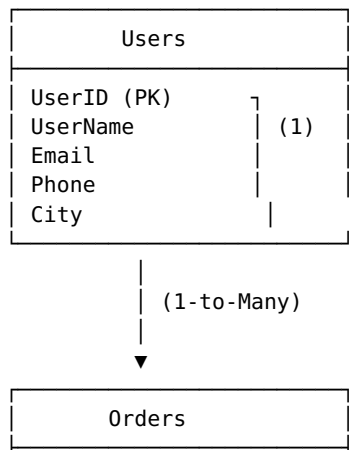
A simplified **E-Commerce Platform** database with: - **Users:** Customers who buy products - **Products:** Items available for sale - **Orders:** Customer purchases - **OrderItems:** Individual items in each order

Why These 4 Tables?

- Shows **1-to-Many relationships** (User → Orders)
 - Shows **Many-to-Many relationships** (via OrderItems)
 - Real-world scenario
 - Perfect for learning JOINS and aggregations
 - Foundation for more complex designs
-

Database Schema (Visual)

Entity Relationship Diagram (ERD):



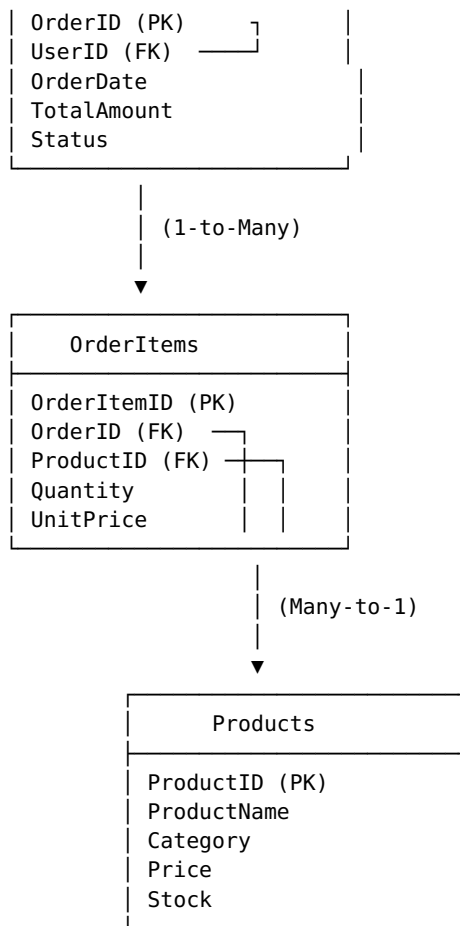


Table Structure & Sample Data

1 USERS Table

Who are the customers?

UserID	UserName	Email	Phone	City
-----	-----	-----	-----	-----
1	Raj Kumar	raj@email.com	9876543210	Delhi
2	Priya Singh	priya@email.com	9876543211	Mumbai
3	Arjun Patel	arjun@email.com	9876543212	Bangalore
4	Neha Sharma	neha@email.com	9876543213	Hyderabad
5	Vikram Verma	vikram@email.com	9876543214	Chennai
6	Anjali Reddy	anjali@email.com	9876543215	Pune
7	Rohit Kumar	rohit@email.com	9876543216	Kolkata
8	Divya Nair	divya@email.com	9876543217	Ahmedabad
9	Sanjay Singh	sanjay@email.com	9876543218	Jaipur
10	Pooja Gupta	pooja@email.com	9876543219	Lucknow

2 PRODUCTS Table

What items are available for sale?

ProductID	ProductName	Category	Price	Stock
-----	-----	-----	-----	-----

1	Samsung Galaxy S23	Electronics	79999	50
2	iPhone 15 Pro	Electronics	129999	30
3	Sony WH-1000XM5	Electronics	29999	100
4	Boat Airdopes 131	Electronics	2999	200
5	Dell XPS 13	Electronics	99999	25
6	MacBook Pro 16"	Electronics	249999	15
7	Nike Air Force 1	Fashion	8999	150
8	Adidas Ultraboost 22	Fashion	11999	120
9	Ray-Ban Wayfarer	Fashion	12999	80
10	Tommy Hilfiger T-Shirt	Fashion	3999	200
11	Levi's 501 Jeans	Fashion	5999	180
12	Zara Winter Jacket	Fashion	15999	60
13	Harman Kardon Speaker	Electronics	49999	40
14	Canon EOS M6	Electronics	64999	20
15	GoPro Hero 12	Electronics	44999	35
16	Kindle Paperwhite	Electronics	13999	90
17	Apple Watch Series 9	Electronics	41999	45
18	JBL Flip 6	Electronics	11999	110
19	Puma Running Shoes	Fashion	7999	140
20	Allen Solly Formal Shirt	Fashion	2999	250
21	Fossil Smartwatch	Electronics	14999	50
22	Calvin Klein Wallet	Fashion	4999	160

3 ORDERS Table

When did customers place orders?

OrderID	UserID	OrderDate	TotalAmount	Status
-----	-----	-----	-----	-----
1	1	2025-05-01	82998	Delivered
2	2	2025-05-02	131998	Delivered
3	3	2025-05-03	29999	Delivered
4	4	2025-05-04	54998	Pending
5	5	2025-05-05	12998	Delivered
6	1	2025-05-06	20998	Shipped
7	2	2025-05-07	99999	Delivered
8	3	2025-05-08	49999	Pending
9	6	2025-05-09	64999	Delivered
10	7	2025-05-10	11999	Shipped
11	4	2025-05-11	42998	Delivered
12	8	2025-05-12	249999	Delivered
13	5	2025-05-13	89998	Pending
14	1	2025-05-14	12999	Delivered
15	9	2025-05-15	34998	Shipped
16	10	2025-05-16	129999	Delivered
17	2	2025-05-17	64999	Pending
18	3	2025-05-18	44999	Delivered
19	6	2025-05-19	22998	Delivered
20	8	2025-05-20	15999	Shipped

4 ORDERITEMS Table

What products are in each order?

OrderItemID	OrderID	ProductID	Quantity	UnitPrice
-----	-----	-----	-----	-----
1	1	1	1	79999

2	1	4	1	2999
3	2	2	1	129999
4	3	3	1	29999
5	4	5	1	99999
6	4	7	1	8999
7	5	8	1	11999
8	5	10	1	3999
9	6	9	1	12999
10	6	11	1	5999
11	7	6	1	249999
12	8	13	1	49999
13	9	14	1	64999
14	10	3	1	29999
15	11	17	1	41999
16	11	18	1	11999
17	12	6	1	249999
18	13	2	1	129999
19	13	4	2	2999
20	14	16	1	13999

CREATE TABLE Statements

Step 1: Create USERS Table

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY IDENTITY(1,1),
    UserName NVARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Phone VARCHAR(15),
    City NVARCHAR(50)
);
```

Explanation: - UserID → Auto-incrementing Primary Key - UserName → Not null, required field - Email → Unique (no duplicate emails), required - Phone → Optional field - City → Customer's city

Step 2: Create PRODUCTS Table

```
CREATE TABLE Products (
    ProductID INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(200) NOT NULL,
    Category NVARCHAR(50) NOT NULL,
    Price DECIMAL(10,2) NOT NULL CHECK (Price > 0),
    Stock INT NOT NULL CHECK (Stock >= 0)
);
```

Explanation: - ProductID → Auto-incrementing Primary Key - ProductName → Product description - Category → Electronics, Fashion, etc. - Price → Must be > 0 - Stock → Must be >= 0 (can't be negative)

Step 3: Create ORDERS Table

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY IDENTITY(1,1),
    UserID INT NOT NULL,
    OrderDate DATETIME DEFAULT GETDATE(),
    TotalAmount DECIMAL(10,2) NOT NULL,
    Status NVARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
```

Explanation: - OrderID → Auto-incrementing Primary Key - UserID → Foreign Key linking to Users table - OrderDate → Defaults to current date/time - TotalAmount → Total order value - Status → Pending, Shipped, or Delivered - **FK Constraint** → Ensures valid user reference

Step 4: Create ORDERITEMS Table

```
CREATE TABLE OrderItems (  
    OrderItemID INT PRIMARY KEY IDENTITY(1,1),  
    OrderID INT NOT NULL,  
    ProductID INT NOT NULL,  
    Quantity INT NOT NULL CHECK (Quantity > 0),  
    UnitPrice DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),  
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)  
);
```

Explanation: - OrderItemID → Auto-incrementing Primary Key - OrderID → Foreign Key to Orders - ProductID → Foreign Key to Products - Quantity → Must be > 0 - UnitPrice → Price at time of purchase - **Two FK Constraints** → Links to both Orders and Products

INSERT Sample Data

Insert USERS (10 rows)

```
INSERT INTO Users (UserName, Email, Phone, City) VALUES  
( 'Raj Kumar', 'raj@email.com', '9876543210', 'Delhi'),  
( 'Priya Singh', 'priya@email.com', '9876543211', 'Mumbai'),  
( 'Arjun Patel', 'arjun@email.com', '9876543212', 'Bangalore'),  
( 'Neha Sharma', 'neha@email.com', '9876543213', 'Hyderabad'),  
( 'Vikram Verma', 'vikram@email.com', '9876543214', 'Chennai'),  
( 'Anjali Reddy', 'anjali@email.com', '9876543215', 'Pune'),  
( 'Rohit Kumar', 'rohit@email.com', '9876543216', 'Kolkata'),  
( 'Divya Nair', 'divya@email.com', '9876543217', 'Ahmedabad'),  
( 'Sanjay Singh', 'sanjay@email.com', '9876543218', 'Jaipur'),  
( 'Pooja Gupta', 'pooja@email.com', '9876543219', 'Lucknow');
```

Insert PRODUCTS (22 rows)

```
INSERT INTO Products (ProductName, Category, Price, Stock) VALUES  
( 'Samsung Galaxy S23', 'Electronics', 79999, 50),  
( 'iPhone 15 Pro', 'Electronics', 129999, 30),  
( 'Sony WH-1000XM5', 'Electronics', 29999, 100),  
( 'Boat Airdopes 131', 'Electronics', 2999, 200),  
( 'Dell XPS 13', 'Electronics', 99999, 25),  
( 'MacBook Pro 16"', 'Electronics', 249999, 15),  
( 'Nike Air Force 1', 'Fashion', 8999, 150),  
( 'Adidas Ultraboost 22', 'Fashion', 11999, 120),  
( 'Ray-Ban Wayfarer', 'Fashion', 12999, 80),  
( 'Tommy Hilfiger T-Shirt', 'Fashion', 3999, 200),  
( 'Levi's 501 Jeans', 'Fashion', 5999, 180),  
( 'Zara Winter Jacket', 'Fashion', 15999, 60),  
( 'Harman Kardon Speaker', 'Electronics', 49999, 40),  
( 'Canon EOS M6', 'Electronics', 64999, 20),  
( 'GoPro Hero 12', 'Electronics', 44999, 35),  
( 'Kindle Paperwhite', 'Electronics', 13999, 90),  
( 'Apple Watch Series 9', 'Electronics', 41999, 45),  
( 'JBL Flip 6', 'Electronics', 11999, 110),
```

```
('Puma Running Shoes', 'Fashion', 7999, 140),
('Allen Solly Formal Shirt', 'Fashion', 2999, 250),
('Fossil Smartwatch', 'Electronics', 14999, 50),
('Calvin Klein Wallet', 'Fashion', 4999, 160);
```

Insert ORDERS (20 rows)

```
INSERT INTO Orders (UserID, OrderDate, TotalAmount, Status) VALUES
(1, '2025-05-01', 82998, 'Delivered'),
(2, '2025-05-02', 131998, 'Delivered'),
(3, '2025-05-03', 29999, 'Delivered'),
(4, '2025-05-04', 54998, 'Pending'),
(5, '2025-05-05', 12998, 'Delivered'),
(1, '2025-05-06', 20998, 'Shipped'),
(2, '2025-05-07', 99999, 'Delivered'),
(3, '2025-05-08', 49999, 'Pending'),
(6, '2025-05-09', 64999, 'Delivered'),
(7, '2025-05-10', 11999, 'Shipped'),
(4, '2025-05-11', 42998, 'Delivered'),
(8, '2025-05-12', 249999, 'Delivered'),
(5, '2025-05-13', 89998, 'Pending'),
(1, '2025-05-14', 12999, 'Delivered'),
(9, '2025-05-15', 34998, 'Shipped'),
(10, '2025-05-16', 129999, 'Delivered'),
(2, '2025-05-17', 64999, 'Pending'),
(3, '2025-05-18', 44999, 'Delivered'),
(6, '2025-05-19', 22998, 'Delivered'),
(8, '2025-05-20', 15999, 'Shipped');
```

Insert ORDERITEMS (20 rows)

```
INSERT INTO OrderItems (OrderID, ProductID, Quantity, UnitPrice)
VALUES
(1, 1, 1, 79999),
(1, 4, 1, 2999),
(2, 2, 1, 129999),
(3, 3, 1, 29999),
(4, 5, 1, 99999),
(4, 7, 1, 8999),
(5, 8, 1, 11999),
(5, 10, 1, 3999),
(6, 9, 1, 12999),
(6, 11, 1, 5999),
(7, 6, 1, 249999),
(8, 13, 1, 49999),
(9, 14, 1, 64999),
(10, 3, 1, 29999),
(11, 17, 1, 41999),
(11, 18, 1, 11999),
(12, 6, 1, 249999),
(13, 2, 1, 129999),
(13, 4, 2, 2999),
(14, 16, 1, 13999);
```

Day 2 Preview - Queries You'll Learn

Query 1: All Orders by Each Customer

```
SELECT
    u.UserName,
```

```
        u.City,  
        o.OrderID,  
        o.OrderDate,  
        o.TotalAmount,  
        o.Status  
FROM Users u  
INNER JOIN Orders o ON u.UserID = o.UserID  
ORDER BY u.UserName, o.OrderDate;
```

Query 2: Products in Each Order

```
SELECT  
    o.OrderID,  
    u.UserName,  
    p.ProductName,  
    p.Category,  
    oi.Quantity,  
    oi.UnitPrice,  
    (oi.Quantity * oi.UnitPrice) AS LineTotal  
FROM Orders o  
INNER JOIN Users u ON o.UserID = u.UserID  
INNER JOIN OrderItems oi ON o.OrderID = oi.OrderID  
INNER JOIN Products p ON oi.ProductID = p.ProductID  
ORDER BY o.OrderID;
```

Query 3: Top 5 Best-Selling Products

```
SELECT TOP 5  
    p.ProductID,  
    p.ProductName,  
    p.Category,  
    SUM(oi.Quantity) AS TotalSold,  
    p.Price  
FROM Products p  
INNER JOIN OrderItems oi ON p.ProductID = oi.ProductID  
GROUP BY p.ProductID, p.ProductName, p.Category, p.Price  
ORDER BY TotalSold DESC;
```

Query 4: Total Revenue Per Customer

```
SELECT  
    u.UserName,  
    u.City,  
    COUNT(DISTINCT o.OrderID) AS TotalOrders,  
    SUM(o.TotalAmount) AS TotalSpent,  
    AVG(o.TotalAmount) AS AvgOrderValue  
FROM Users u  
LEFT JOIN Orders o ON u.UserID = o.UserID  
GROUP BY u.UserID, u.UserName, u.City  
ORDER BY TotalSpent DESC;
```

What You'll Learn

Day 1:

- Create these 4 tables
- Insert 52+ rows of data
- Basic SELECT queries
- Understand the schema

Day 2:

INNER JOINS (multiple tables)
Aggregate functions (COUNT, SUM, AVG)
GROUP BY queries
Real business questions
ORDER BY and filtering

Summary: Your E-Commerce Database

Table	Rows	Purpose
Users	10	Store customer information
Products	22	Catalog of items for sale
Orders	20	Customer purchase records
OrderItems	20	Individual items in orders

Total Data: 72 rows across 4 interconnected tables

Ready to Build?

Copy all the CREATE and INSERT statements and run them in SQL Server Management Studio!

After this, you'll have a real database you can query for business insights!

Happy Building!